CP493 Directed Research

Progress Report: May 1 – June 22, 2026

Ranjot Sandhu · Wilfrid Laurier University

RL Control of a 2-DOF Robotic Arm (Fischer et al. 2021)

Executive Summary

Over the past seven weeks the project advanced from a working single-target SAC baseline into a full waypoint-navigation system with biomechanically realistic muscle actuation, automated multi-seed hyperparameter search, and an adaptive hold curriculum. The arm can now be trained to visit three specific workspace targets in sequence and hold the final one stably — the core deliverable for Phase 1 of the project.

Week 1 — May 1–9: Fischer Integration Steps 1–5

Goal: Implement the full Fischer et al. (2021) training protocol on top of the existing PPO baseline.

What was done

Step	Commit	Description
Step 1	`77b7180`	Added SAC as the default algorithm; made goal tolerance
Step 2	`9d31252`	Implemented `AdaptiveCurriculumCallback` — rolling 50-e
Step 2	`9d31252`	Added motor babbling initialisation — random joint angles
Step 3	`6f995f0`	Built `FittsLawValidator` — sweeps 12 target amplitudes ×
Step 4	`5f37a26`	Built `PowerLawValidator` — runs 20 circular arc trajector
Step 5	`5f37a26`	Implemented Hill-type muscle actuation — `MuscleModel`
Launcher	`7431988`	Added unified GUI launcher (`python -m rl_armMotion.two

Key technical decisions

- SAC chosen over PPO because SAC's entropy regularisation and off-policy replay buffer are essential for learning the precise hold behaviour (20 consecutive steps within position + orientation + velocity tolerances). PPO was confirmed unable to solve the task.
- Muscle actuation chosen because it produces biologically plausible smooth trajectories from the force-velocity dynamics alone, without extra smoothness penalty terms in the reward.

Results

- SAC with adaptive curriculum and motor babbling reliably reaches EAST goal in ~200K steps.
- Fitts' Law regression achieved $R^2 > 0.85$ on trained policy.

Week 2 — May 9–19: Muscle Mode Completion + Click-to-Pick Goals

Goal: Surface muscle actuation in the GUI and add flexible goal-placement.

What was done

Commit	Description
`5f37a26`	Completed Hill-type muscle dynamics with configurable maximum isometric forc
`5c4b822`	Documented the unified GUI launcher in README
`0600202`	Added **Click-to-Pick goal mode** to Training GUI — user clicks anywhere on th
`0600202`	Intermediate waypoints advance on touch (position within tolerance); final waypo
`a570ebb`	Updated progress document and Reward System PDF for Phase 6 planning

Key technical detail — waypoint sequencing

The `ArmTaskEnv.set_waypoints()` method accepts an ordered list of `[x, y]` coordinates. During an episode the agent receives the current waypoint's position in its observation. When the hold condition is met for an intermediate waypoint the environment advances `current_waypoint_index` and resets the hold counter. The final waypoint requires the full 20-step hold for episode termination (success). This required no changes to the SAC hyperparameters.

Week 3 — May 19–27: Phase 6 Complete + GUI Polish

Goal: Complete muscle actuation integration end-to-end and harden the system.

What was done

Commit	Description
`ce2d617`	Surfaced Phase 6 muscle actuation in the Training GUI — actuation mode selec
`cc2c1d7`	Added curriculum metrics panel to Training GUI — live display of current toleran
`cc2c1d7`	Added **Run Validators** button to Training GUI — triggers Fitts' Law and 2/3 P
`8489c8d`	Fixed Fischer validator plot crash on macOS due to matplotlib being called from
`da8812e`	Fixed subprocess detachment — spawned training processes no longer die whe

Commit	Description
`62126cc`	Updated handoff document (`progress.md`) for Phase 6 completion

System state at end of Week 3

- Full Fischer 2021 protocol operational end-to-end: SAC + adaptive curriculum + motor babbling + Hill-type muscle actuation + Fitts'/Power Law validators
- GUI allows non-technical users to train, visualise, save, and validate a model without touching the command line

---

Week 4 — May 27 – June 17: Code Review + Documentation Pass

Goal: Prepare the codebase for external review (professor evaluation).

What was done

Commit	Description
`f4767f8`	Code review phase 1 — removed unused imports across all modules; clarified s
`39b7c49`	Code review phase 2 — deduplicated goal-state mutation in `ArmTaskEnv`; goa
`7d056da`	Expanded README installation section with a full GitHub-friendly setup guide (p
`5bd4aba`	Added `INSTALL.md` — standalone install guide for users who prefer a dedicate
`8673cfc`	Updated `.gitignore` — added Claude Code worktrees, model checkpoints, and

Code quality improvements

- All public classes and functions have module-level docstrings explaining the `what`, `why`, and parameters.
- Reward function documented term-by-term (10 reward components, each with its purpose and weight).
- Curriculum callback documented with the four preconditions that must hold before a tolerance decay fires.

Week 5 — June 17–19: Training CLI + Evaluation Harness

Goal: Enable headless (no-GUI) training runs for long batch searches.

What was done

Commit	Description
`585caf1`	Added `--actuation-mode` flag to `train_fischer_session.py` — can now specify `
`42f101f`	Fixed arm-control GUI — now auto-detects the actuation mode of a loaded mod

Commit	Description
`53cfb94`	Added `--waypoints` flag to `train_fischer_session.py` — CLI training can now ta
`e2c0ef4`	Added `RESULTS.md` — empirical numbers from the 200K muscle-mode SAC
`bf363d1`	Added evaluation harness — scripts that load a saved model and run determinis
`6182188`	Added 500K training results — full training artefacts from the first 500K-step mus
`4e398a2`	Added Training Results Report PDF for professor review

Key result from 500K muscle-mode run

- Mean episode reward: **22,187**
- Best episode reward: **25,718**
- Training time: **33 minutes** (500K steps on a 24-core Windows machine)
- The model consistently reaches waypoints 1 and 2 but struggles to hold at waypoint 3 — establishing the core research problem for the next phase

---

Week 6 — June 19–21: Documentation + Final Pre-Training Polish

Goal: Full inline documentation pass; finalize code before batch search.

What was done

Commit	Description
`0b2d810`	Expanded inline documentation for all Fischer integration files — every non-obvi
`3722bbb`	Added class-level docstring to `RLTrainerWithMetrics` explaining its two roles: m
`2fcb884` / `d2e84b9`	Final code cleanup before launching parallel training — removed debug prints, s

---

Week 7 — June 21–22 (Current): Parallel Seed Search + Hold Curriculum

Goal: Find a model that achieves 3/3 waypoints reliably, then study and preserve it.

What was built

`run_auto_search.py` — Automated batch seed search:

- Runs 5 seeds in parallel, each for 500K steps
- At 250K steps: kills any seed with reward < 0 (saves ~45 minutes per bad seed)
- After each batch: evaluates all completed models deterministically
- When a 3/3 winner is found: runs 5 consistency checks, evaluates at 3 tolerances (0.6 m / 0.4 m / 0.2 m), copies model to `CHAMPION/`, generates `REPLICATION_REPORT.md`

- Each seed writes to its own `stdout.log` so progress can be monitored live

`HoldCurriculumCallback` — Adaptive hold requirement:

- Added to `curriculum_callback.py` alongside the existing tolerance curriculum
- Starts `hold_steps_required = 10` (instead of hardcoded 20)
- Graduates to 15, then 20, as the rolling 50-episode success rate exceeds 60%
- Motivation: the agent previously learned to reach waypoints 1 and 2 reliably but could not master the 20-step hold at waypoint 3. Starting with an easier hold (10 steps) gives the agent a learning signal before demanding full precision.
- Hold decrement reduced from -5 to -3: a brief slip out of the goal zone now costs 3 steps of progress instead of 5, making the hold requirement less brittle.

Training progress as of June 22

- Batches 1–2 (seeds 001–010): completed, no 3/3 winner — all reached 2/3 waypoints with high rewards (24K–29K), establishing that the arm learns the trajectory but fails only at the final hold
- Batch 3 (seeds 011–015): currently training with the new hold curriculum — 2.4% complete
- Architecture: 5 parallel seeds, 3 PyTorch threads per seed, early kill at 250K steps

---

Summary of All Deliverables

Deliverable	Status	Location
SAC + adaptive curriculum (Fischer 2021 protocol)	Complete	`src/rl_armMotion/two_d/training/curriculum_callback.py`
Hill-type muscle actuation	Complete	`src/rl_armMotion/two_d/utils/muscle_model.py`
Fitts' Law validator	Complete	`src/rl_armMotion/two_d/validation/fitts_law.py`
2/3 Power Law validator	Complete	`src/rl_armMotion/two_d/validation/power_law.py`
Waypoint navigation (3 targets)	Complete	`src/rl_armMotion/two_d/environments/task_env.py`
Training Dashboard GUI (muscle mode + curriculum panel)	Complete	`src/rl_armMotion/two_d/gui/training_gui.py`
Interactive Arm GUI (model simulation)	Complete	`src/rl_armMotion/two_d/gui/app.py`
Headless training CLI	Complete	`scripts/train_fischer_session.py`
Automated parallel seed search	Complete	`run_auto_search.py`
Hold curriculum (adaptive hold requirement)	Complete	`src/rl_armMotion/two_d/training/curriculum_callback.py`
Fischer Implementation Report (PDF)	Complete	`docs/Fischer_Implementation_Report.pdf`
Training Results Report (PDF)	Complete	`docs/Training_Results_Report.pdf`
3/3 waypoint champion model	**In progress**	`project_assets/outputs/CHAMPION/` (when found)

Planned Next Phases

Phase 2 — Generalised Waypoints + Human ROM Constraints

After a 3/3 champion is found, the environment will be extended to:

- Generate random waypoints anywhere in the reachable workspace
- Enforce human range-of-motion limits (shoulder: approximately –60° to +180° flexion/extension; elbow: 0° to 120°)
- Support right-arm and left-arm configurations (mirrored joint limits)
- Train from scratch on the generalised task (~500K–1M steps)

Phase 3 — Five-Waypoint Navigation

Extend to 5-waypoint sequences. May require increased episode length budget and adjusted curriculum thresholds. Model to be trained from scratch (never fine-tuned from Phase 2).

Long-term — Real Arm Controller

The trained policy maps directly to muscle activation commands (0–1 per muscle), making it suitable as a forward model for a physical prosthetic or rehabilitation device controller.

Report generated: 2026-06-22

Project repository: `C:\Users\ranjo\OneDrive\Desktop\ARM-Model`